Keccak Table

목적

Keccak Table은 **Keccak256 해시 연산의 입력과 출력을 저장**하는 테이블입니다. 이 테이블은 Keccak Circuit에서 생성되며, 여러 회로에서 해시 검증에 사용됩니다. Keccak256은 이더리움에서 가장 널리 사용되는 해시 함수로, 주소 생성, 트랜잭션 해시, 스토리지 키 계산 등에 사용됩니다.

테이블 스키마

파일: `zkevm-circuits/src/table.rs` (line 1439)

```
pub struct KeccakTable {
/// 행 활성화 여부
pub q_enable: Column<Fixed>,
/// 최종 행 여부
pub is_final: Column<Advice>,
/// 입력 RLC (역순)
pub input_rlc: Column<Advice>,
/// 입력 길이
pub input_len: Column<Advice>,
/// 출력 RLC
pub output_rlc: Column<Advice>,
```

```
}
```

컬럼 설명

1. q_enable (Fixed)

- 해당 행이 유효한 Keccak 행인지 표시
- 모든 Keccak 행에서 `1`

2. is_final (Advice)

- 해시 계산의 최종 행인지 표시
- `1` : 최종 행 (완전한 해시 결과)
- `0` : 중간 행 (부분 해시)

3. input_rlc (Advice, SecondPhase)

- 입력 바이트 배열의 RLC 인코딩
- **역순**: `RLC(reversed(input))`
- Challenge를 사용하여 SecondPhase에 할당

4. input_len (Advice)

- 입력 바이트 배열의 길이
- 0부터 임의의 길이까지 가능

5. output_rlc (Advice, SecondPhase)

- 해시 결과(32바이트)의 RLC 인코딩
- Challenge를 사용하여 SecondPhase에 할당

RLC 인코딩

Keccak Table은 **RLC (Random Linear Combination)**를 사용하여 바이트 배열을 압축합니다.

input_rlc 계산

입력 바이트 배열을 **역순**으로 RLC 인코딩합니다:

```
// 입력: [b0, b1, b2, b3]

// input_rlc = b3 + b2 * r + b1 * r^2 + b0 * r^3

input_rlc = challenges.keccak_input().map(|challenge|

rlc::value(input.iter().rev(), challenge)

);
```

역순 사용 이유:

- Keccak Circuit의 내부 처리 방식과 일치
- 효율적인 검증 가능

output_rlc 계산

출력 해시(32바이트)를 Little-Endian으로 RLC 인코딩합니다:

```
// 해시 결과: 32바이트

let  output = keccak256(input);


// Little-Endian으로 변환 후 RLC

output_rlc = challenges.evm_word().map(|challenge|

rlc::value(&Word::from_big_endian(output).to_le_bytes(), challenge)
```

```
);
```

테이블 레이아웃

기본 구조

각 Keccak 해시 연산은 하나의 행으로 표현됩니다:

```
q_enable | is_final | input_rlc | input_len | output_rlc
---------|----------|-----------|-----------|------------
1 | 0 | 0 | 0 | 0 (All-zero 행)
1 | 1 | rlc1 | 4 | hash1_rlc
1 | 1 | rlc2 | 32 | hash2_rlc
1 | 1 | rlc3 | 100 | hash3_rlc
...
```

All-Zero 행

첫 번째 행은 항상 all-zero 행입니다:

- 빈 입력에 대한 해시를 나타냄
- 또는 패딩용

예시: 간단한 해시

입력: `0x12345678` (4바이트)

```
q_enable | is_final | input_rlc | input_len | output_rlc
```

```
---------|----------|-----------------------------|-----------|----------------
1 | 1 | 0x78 + 0x56*r + 0x34*r^2 + 0x12*r^3 | 4 | keccak256(0x12345678)
```

사용 사례

Keccak Table은 다양한 회로에서 사용됩니다.

1. Bytecode Circuit

바이트코드의 code hash 검증:

```
// Bytecode Circuit에서 Keccak Table lookup
meta.lookup_any("bytecode keccak", |meta| {
vec![
1.expr(), // q_enable
1.expr(), // is_final
bytecode_rlc.expr(), // input_rlc
bytecode_length.expr(), // input_len
code_hash_rlc.expr(), // output_rlc
]
});
```

검증 내용:

- `keccak256(bytecode) == code_hash`

2. Tx Circuit

트랜잭션 해시 검증:

```
// Tx Circuit에서 Keccak Table lookup
meta.lookup_any("tx hash keccak", |meta| {
vec![
1.expr(), // q_enable
1.expr(), // is_final
tx_rlp_rlc.expr(), // input_rlc (RLP 인코딩)
tx_rlp_length.expr(), // input_len
tx_hash_rlc.expr(), // output_rlc
]
});
```

검증 내용:

- `keccak256(rlp_encode(tx)) == tx_hash`

3. Sig Circuit

서명 메시지 해시 계산:

```
// Sig Circuit에서 Keccak Table lookup
meta.lookup_any("sig msg hash", |meta| {
vec![
1.expr(), // q_enable
1.expr(), // is_final
msg_rlc.expr(), // input_rlc
```

```
msg_length.expr(), // input_len

msg_hash_rlc.expr(), // output_rlc

]

});
```

검증 내용:

- `keccak256(message) == msg_hash`
- ECDSA 서명 검증에 사용

4. EVM Circuit

CREATE/CREATE2 opcode에서 주소 계산:

```
// CREATE: address = keccak256(rlp([sender, nonce]))[-20:]

// CREATE2: address = keccak256(0xff ++ sender ++ salt ++ keccak256(init_code))[
```

```
meta.lookup_any("create address", |meta| {

vec![

1.expr(), // q_enable

1.expr(), // is_final

create_input_rlc.expr(), // input_rlc

create_input_len.expr(), // input_len

address_hash_rlc.expr(), // output_rlc

]

});
```

```
// 주소는 해시의 마지막 20바이트
let  address = address_hash_rlc & 0xFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF;
```

5. State Circuit

계정 주소에서 스토리지 키 계산:

```
// 스토리지 키 = keccak256(key ++ address)
meta.lookup_any("storage key", |meta| {
vec![
1.expr(), // q_enable
1.expr(), // is_final
key_address_rlc.expr(), // input_rlc
32 + 20, // input_len (key: 32, address: 20)
storage_key_rlc.expr(), // output_rlc
]
});
```

Keccak Circuit과의 연동

Keccak Circuit은 Keccak Table에 데이터를 할당하고 해시를 계산합니다.

데이터 할당

파일: `zkevm-circuits/src/table.rs` (line 1485)

```
impl KeccakTable {
pub fn assignments<F: Field>(
input: &[u8],
challenges: &Challenges<Value<F>>,
) -> Vec<[Value<F>; 4]> {
// 1. input_rlc 계산 (역순)
let input_rlc = challenges
.keccak_input()
.map(|challenge| rlc::value(input.iter().rev(), challenge));
// 2. input_len 계산
let input_len = F::from(input.len() as u64);
// 3. 해시 계산
let output = sha3::Keccak256::digest(input);
// 4. output_rlc 계산 (Little-Endian)
let output_rlc = challenges.evm_word().map(|challenge| {
rlc::value(
&Word::from_big_endian(output.as_slice()).to_le_bytes(),
challenge,
)
});
vec![[
```

```
Value::known(F::one()), // is_final
input_rlc,
Value::known(input_len),
output_rlc,
]]
}
}
```

dev_load 메서드

테스트 및 개발용 데이터 로드:

```
impl KeccakTable {
pub fn dev_load<'a, F: Field>(
&self,
layouter: &mut impl Layouter<F>,
inputs: impl IntoIterator<Item = &'a Vec<u8>> + Clone,
challenges: &Challenges<Value<F>>,
) -> Result<(), Error> {
layouter.assign_region(
|| "keccak table",
|mut region| {
let mut offset = 0;
// All-zero 행
```

```
for column in <KeccakTable as LookupTable<F>>::advice_columns(self) {
region.assign_fixed(
|| "keccak table all-zero row",
self.q_enable,
offset,
|| Value::known(F::one()),
)?;
region.assign_advice(
|| "keccak table all-zero row",
column,
offset,
|| Value::known(F::zero()),
)?;
}
offset += 1;


// 각 입력에 대해 행 할당
for input in inputs.clone() {
for row in Self::assignments(input, challenges) {
region.assign_fixed(
|| format!("keccak table row {offset}"),
self.q_enable,
offset,
```

```
|| Value::known(F::one()),
)?;
// is_final, input_rlc, input_len, output_rlc 할당
offset += 1;
}
}
Ok(())
},
)
}
}
```

성능 최적화

1. RLC 압축

- 입력과 출력을 RLC로 압축하여 컬럼 수 감소
- 32바이트 해시를 하나의 필드 원소로 표현

2. SecondPhase 컬럼

RLC를 사용하는 컬럼들은 SecondPhase에 할당:

- `input_rlc`
- `output_rlc`

3. Challenge 사용

- `keccak_input()` : 입력 RLC용 challenge
- `evm_word()` : 출력 RLC용 challenge

제약 조건

1. 해시 정확성

```
// Keccak Circuit에서 검증
// keccak256(input) == output
```

2. RLC 일관성

```
// input_rlc는 입력 바이트의 역순 RLC
// output_rlc는 해시 결과의 Little-Endian RLC
```

3. is_final 플래그

```
// is_final == 1: 완전한 해시 결과
// is_final == 0: 중간 상태 (사용되지 않음)
```

예시: 전체 해시 연산

예시 1: 빈 입력

입력: `[]` (0바이트)

해시: `0xc5d2460186f7233c927e7db2dcc703c0e500b653ca82273b7bfad8045d85a470`

Keccak Table:

```
q_enable | is_final | input_rlc | input_len | output_rlc

---------|----------|-----------|-----------|------------------

1 | 1 | 0 | 0 | hash_rlc(empty)
```

예시 2: 간단한 입력

입력: `0x1234` (2바이트)

해시: `keccak256(0x1234)`

Keccak Table:

```
q_enable | is_final | input_rlc | input_len | output_rlc

---------|----------|--------------------| ----------|------------------

1 | 1 | 0x34 + 0x12 * r | 2 | hash_rlc(0x1234)
```

예시 3: 트랜잭션 해시

트랜잭션 RLP: `0xf86c...` (108바이트)

Keccak Table:

```
q_enable | is_final | input_rlc | input_len | output_rlc

---------|----------|----------------------------|-----------|------------------

1 | 1 | rlc(reversed(tx_rlp)) | 108 | tx_hash_rlc
```

예시 4: CREATE 주소 계산

입력: `rlp([sender, nonce])`

Solidity:

```
address newContract = address(uint160(uint256(

keccak256(abi.encodePacked(

bytes1(0xd6), // RLP prefix

bytes1(0x94), // Address prefix

msg.sender,

bytes1(0x01) // Nonce = 1

))

)));
```

Keccak Table:

```
q_enable | is_final | input_rlc | input_len | output_rlc

---------|----------|---------------------------|-----------|------------------

1 | 1 | rlc(reversed(rlp_input)) | 23 | address_hash_rlc
```

주소 추출:

```
// 해시의 마지막 20바이트

let  address = address_hash & 0xFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF;
```

디버깅 팁

Keccak 해시 검증

```
// Rust에서 Keccak256 계산
use sha3::{Digest, Keccak256};

let input = vec![0x12, 0x34];
let output = Keccak256::digest(&input);
println!("Hash: 0x{}", hex::encode(output));
```

RLC 계산 검증

```
// input_rlc 계산 (역순)
let input = vec![0x12, 0x34];
let challenge = F::from(12345); // 예시 challenge

let mut rlc = F::zero();
for byte in input.iter().rev() {
rlc = rlc * challenge + F::from(*byte as u64);
}
println!("input_rlc: {:?}", rlc);
```

일반적인 오류

1. **해시 불일치**: 입력과 출력 해시가 맞지 않음
2. **RLC 계산 오류**: 역순 또는 Little-Endian 처리 오류
3. **Challenge 불일치**: 잘못된 challenge 사용

Keccak256 vs SHA256

Keccak256 (이더리움 표준)

- **사용**: 주소, 트랜잭션 해시, 스토리지 키 등
- **출력**: 32바이트
- **Opcode**: `SHA3` (0x20) - 실제로는 Keccak256
- **테이블**: Keccak Table

SHA256 (프리컴파일)

- **사용**: SHA256 프리컴파일 (0x02)
- **출력**: 32바이트
- **테이블**: SHA256 Table (별도)

주의: Solidity의 `keccak256()` 은 실제로 Keccak256을 사용하며, SHA3-256과는 다릅니다.

관련 문서

- Keccak Circuit - Keccak Table을 생성하는 회로
- Bytecode Circuit - Code hash 검증
- Tx Circuit - 트랜잭션 해시 검증
- Sig Circuit - 서명 메시지 해시
- RLC 기법 - RLC 인코딩 상세

참고 자료

- **코드**: `zkevm-circuits/src/table.rs` (line 1439)
- **Keccak Circuit**: `zkevm-circuits/src/keccak_circuit.rs`
- **Keccak256 표준**: https://keccak.team/keccak.html
- **이더리움 Yellow Paper**: https://ethereum.github.io/yellowpaper/paper.pdf